

École Nationale des Sciences Appliquées d'Oujda

Filière : *Génie des Systèmes Electronique, Informatique et Réseaux
(GSEIR)*

RAPPORT DE STAGE

*Système embarqué à base de STM32 pour le suivi et le
diagnostic des moteurs électriques*

Réalisé par :

Mohamed HRIR

Encadré par :

Tuteur entreprise : FAKKOUS

ABD ELLAH

Période de stage : du 15/07/2026 au 15/08/2026

Année universitaire : 2025 – 2026

Remerciements

Nous tenons tout d'abord à exprimer notre profonde gratitude à Allah, le Tout-Puissant, pour Sa guidance, Sa protection et les bienfaits qu'Il nous accorde depuis le début de notre vie.

C'est avec un immense plaisir que nous adressons nos sincères remerciements à toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce travail.

Nous remercions chaleureusement l'équipe de l'**atelier de bobinage** de l'**OCP Khou-ribga** pour leur accueil, leur disponibilité et les explications qu'ils n'ont cessé de nous fournir durant notre stage.

Nous exprimons particulièrement notre reconnaissance à M. **MRABET KANDRI NOUREDDINE**, chef d'atelier, ainsi qu'à M. **FAKKOUS ABD ELLAH** pour son accompagnement et l'intérêt qu'il/elle a porté à notre formation.

Nos remerciements vont également à l'ensemble des techniciens pour leur collaboration, leur professionnalisme et pour le partage précieux de leurs connaissances et compétences, qui ont largement enrichi notre apprentissage.

Nous adressons enfin nos remerciements aux membres du jury, qui ont accepté d'évaluer ce rapport et de consacrer de leur temps pour l'examiner.

Pour conclure, nous tenons à remercier notre établissement, **École Nationale des Sciences Appliquées d'Oujda**, pour les enseignements de qualité, l'encadrement et les ressources mis à notre disposition, qui ont été essentiels à la réussite de ce projet.

Résumé

Durant mon stage de fin d'études effectué au sein de l'atelier de bobinage de l'OCP Khouribga, spécialisé dans les moteurs électriques, les pompes submersibles et les transformateurs, j'ai eu l'opportunité de travailler sur un projet orienté vers la partie matérielle (hardware) du diagnostic des équipements électromécaniques. Mon projet consiste en la conception d'un système embarqué équipé de trois capteurs permettant de mesurer en temps réel la température, les vibrations et la vitesse de rotation (RPM) des moteurs électriques.

En parallèle, je me suis intéressé au matériel utilisé par les techniciens pour effectuer les mesures de résistance des enroulements et du courant d'isolement, deux paramètres essentiels pour évaluer l'état électrique des moteurs et anticiper d'éventuelles défaillances.

L'ensemble de ces données, qu'elles proviennent des capteurs ou des instruments de mesure des techniciens, est destiné à être centralisé au sein d'une application dédiée. Cette application aura pour rôle d'analyser automatiquement les données collectées et de générer un rapport PDF récapitulatif, facilitant ainsi le suivi de l'état des équipements et la prise de décision en matière de maintenance préventive et corrective.

Table des matières

Remerciements	1
Résumé	2
Introduction générale	7
1 Présentation de l'organisme d'accueil	8
1.1 Fiche d'identité du Groupe OCP	8
1.2 Présentation du site de Khouribga	8
1.3 L'atelier de bobinage	9
1.4 Principales défaillances rencontrées sur les moteurs électriques	10
1.5 Organigramme	10
2 Contexte et cahier des charges	12
2.1 Problématique	12
2.2 Objectifs	12
2.3 Méthodologie	13
3 Travail réalisé	14
3.1 Architecture générale du système	14
3.2 Partie matérielle	15
3.3 Partie logicielle	22
3.4 Résultats obtenus	26
Conclusion générale	28
Bibliographie	29
A Annexes	30
Annexes	30
A.1 Boucle principale (<code>main.c</code>)	30

A.2	Communication UART (<code>uart.c</code>)	32
A.3	Traitement du signal de vibration (<code>Vibration.c</code>)	33
A.4	Protocole applicatif (<code>protocol.c</code>)	35

Table des figures

1.1	Vue de l'atelier de bobinage de l'OCP Kouribga	9
1.2	Organigramme du service Maintenance des S/E Électrique & Bobinage (OIK/G/M/B-339)	11
3.1	Architecture générale du système (capteurs + STM32 + application Electron)	14
3.2	Carte STM32 Blue Pill (STM32F103C8T6) utilisée pour le système embarqué	15
3.3	Prototype du système embarqué réalisé (STM32 Blue Pill + capteurs) . . .	16
3.4	Capteurs utilisés : KY-033 (vitesse de rotation), DS18B20 (température), ADXL345 (vibration)	16
3.5	Boîtier du prototype	17
3.6	Emplacement du capteur de température DS18B20 sur le moteur	18
3.7	Emplacement de l'accéléromètre ADXL345 pour la mesure des vibrations .	19
3.8	Positionnement du capteur infrarouge KY-033 face au repère de l'arbre pour la mesure du RPM	19
3.9	Test du prototype sur un moteur réel de l'atelier	20
3.10	Testeur d'isolement Megger MIT1025 utilisé pour la mesure de la résistance d'isolement	21
3.11	Programmeur/débogueur ST-Link utilisé pour flasher le STM32	22
3.12	Écran d'accueil : sélection du port de communication et connexion au mi- crocontrôleur	23
3.13	Écran de saisie des informations du moteur	23
3.14	Tableau de bord d'inspection : capteurs en temps réel, acquisition, mesures électriques et diagnostic	24
3.15	Écran de détails d'une inspection : synthèse des mesures et diagnostic final	24
3.16	Historique des inspections réalisées	25
3.17	PDF généré après l'essai	27

Liste des tableaux

Introduction générale

Dans le cadre de ma formation en génie des systèmes électroniques, informatique et réseaux, j'ai eu l'opportunité d'effectuer mon stage d'initiation au sein de l'atelier de bobinage de l'OCP Khouribga, spécialisé dans la maintenance des moteurs électriques, des pompes industrielles et des transformateurs. Cette expérience m'a permis de découvrir l'environnement professionnel, d'appliquer mes connaissances théoriques sur le terrain et de m'initier aux différentes opérations de diagnostic, de suivi technique et de maintenance des équipements industriels, tout en explorant les possibilités d'intégration de solutions électroniques et embarquées dans ce domaine.

Face à l'importance de la continuité de fonctionnement des machines électriques dans le secteur industriel, la maintenance constitue un enjeu majeur pour garantir la performance, la fiabilité et la sécurité des installations. Les pannes fréquentes, les défauts techniques et l'usure des équipements nécessitent une analyse approfondie ainsi qu'une maîtrise des méthodes de maintenance préventive et corrective, mais aussi le recours à des outils modernes de surveillance et de diagnostic, tels que les systèmes embarqués.

L'objectif principal de ce stage est de découvrir les différentes techniques de maintenance appliquées aux moteurs, pompes et transformateurs, d'identifier les principales défaillances rencontrées, et de proposer une solution électronique embarquée permettant de collecter, analyser et suivre en temps réel les paramètres critiques des moteurs électriques, afin d'améliorer la disponibilité et la durée de vie des équipements.

Chapitre 1

Présentation de l'organisme d'accueil

1.1 Fiche d'identité du Groupe OCP

Le Groupe OCP (Office Chérifien des Phosphates) est une entreprise marocaine à caractère industriel et commercial, créée par Dahir le 7 août 1920. L'activité d'extraction et de traitement du phosphate débute officiellement en 1921, avec l'ouverture de la première mine du groupe dans **l'Atelier de Bobinage à Khouribga**.

Le Groupe OCP est aujourd'hui l'un des leaders mondiaux du marché des phosphates et de leurs dérivés, avec des activités couvrant l'ensemble de la chaîne de valeur : l'extraction de la roche de phosphate, sa transformation chimique en acide phosphorique, ainsi que la production de différents engrais destinés à l'agriculture.

- **Raison sociale** : Office Chérifien des Phosphates (OCP)
- **Date de création** : 7 août 1920
- **Secteur d'activité** : Extraction, transformation et commercialisation du phosphate et de ses dérivés
- **Siège social** : Casablanca
- **Site du stage** : Khouribga – Atelier de bobinage

1.2 Présentation du site de Khouribga

Le site de Khouribga abrite la première mine du Groupe OCP, en exploitation depuis 1921. Il s'agit du principal gisement du groupe, qui concentre l'essentiel des réserves de phosphate du pays. Le phosphate extrait à Khouribga est ensuite acheminé, notamment par pipeline, vers les sites de traitement chimique du groupe.

Pour assurer la continuité de la production, le site dispose de plusieurs unités et ateliers de maintenance dédiés aux différents équipements électromécaniques utilisés dans

le processus d'extraction et de traitement du phosphate (moteurs électriques, pompes, transformateurs, convoyeurs...).

1.3 L'atelier de bobinage

L'atelier de bobinage, dans lequel s'est déroulé notre stage, est spécialisé dans la maintenance préventive et corrective des équipements électromécaniques du site : moteurs asynchrones (MAS), pompes submersibles et transformateurs. Ses missions principales sont :

- Le diagnostic des pannes électriques et mécaniques sur les moteurs et équipements ;
- Le démontage, l'inspection et le contrôle des composants internes (enroulements, roulements, isolants...) ;
- Le rebobinage des moteurs électriques défectueux ;
- Les mesures électriques de contrôle (résistance des enroulements, résistance d'isolement...) ;
- Les essais de fonctionnement après réparation avant remise en service.



FIGURE 1.1 – Vue de l'atelier de bobinage de l'OCP Khouribga

1.4 Principales défaillances rencontrées sur les moteurs électriques

Durant notre stage, nous avons pu observer que les moteurs électriques traités à l'atelier présentent principalement trois types de défaillances :

- **Blocage des roulements à billes** : l'encrassement, l'usure ou le manque de lubrification des roulements peuvent entraîner leur blocage partiel ou total, provoquant des vibrations anormales, un échauffement excessif, voire l'arrêt complet du moteur. Ce type de défaut nécessite un nettoyage, une relubrification ou, si nécessaire, un remplacement des roulements ;
- **Dégradation de la matière isolante des enroulements** : sous l'effet de la chaleur, de l'humidité ou du vieillissement, l'isolant qui recouvre les fils des enroulements peut se détériorer. Cette dégradation entraîne une baisse de la résistance d'isolement, ce qui augmente le risque de court-circuit entre spires ou de défaut à la masse ;
- **Endommagement du bobinage** : lorsque les enroulements sont trop endommagés (court-circuit, coupure, isolant détruit) pour être réparés localement, le moteur doit subir un **rebobinage complet**, c'est-à-dire un remplacement intégral du bobinage.

C'est précisément pour détecter ces défaillances de manière fiable et rapide que les techniciens réalisent les différentes mesures présentées dans ce rapport (résistance des enroulements, résistance d'isolement, vibrations, température, vitesse de rotation), et que le système développé dans ce projet vise à faciliter.

1.5 Organigramme

L'atelier de bobinage fait partie du service **OIK/G/M/B-339 – Maintenance des Systèmes Électriques & Bobinage**, dirigé par le chef de service **M. SALLAOU SAAD**. Ce service est structuré autour d'un chef d'atelier, de plusieurs responsables d'entités (maintenance des moteurs, bobinage, contrôle matériel, gestion HSE, gestion des ressources...), qui encadrent à leur tour différentes sections et équipes techniques.

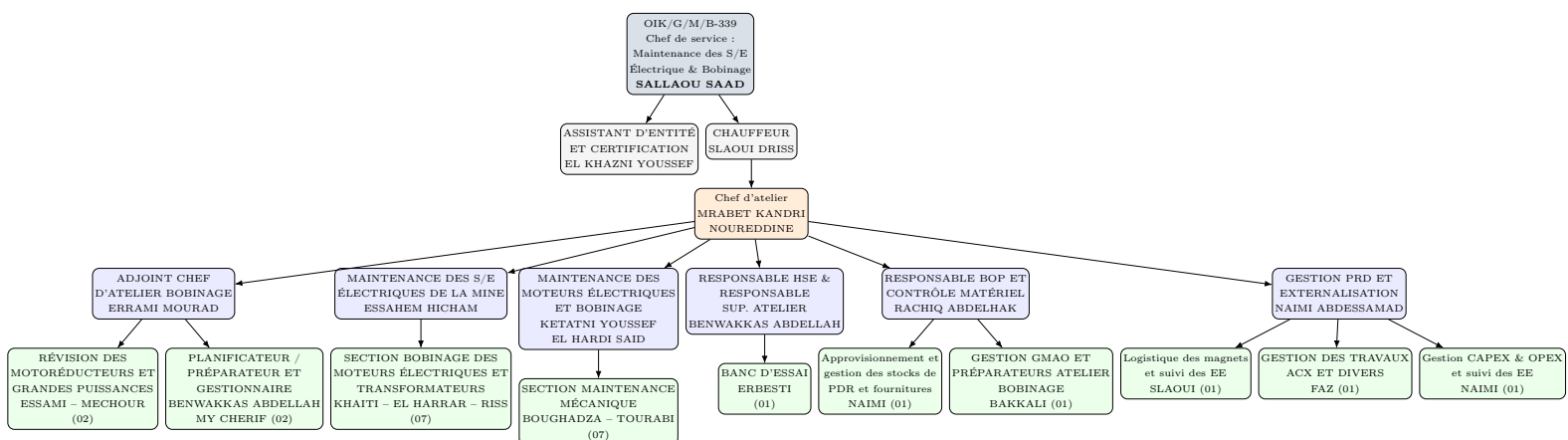


FIGURE 1.2 – Organigramme du service Maintenance des S/E Électrique & Bobinage (OIK/G/M/B-339)

Chapitre 2

Contexte et cahier des charges

2.1 Problématique

Lors des opérations de diagnostic des moteurs électriques, les techniciens de l’atelier de bobinage réalisent plusieurs types de mesures : des mesures manuelles à l’aide d’appareils dédiés (courant par phase, résistance d’isolement), ainsi qu’un suivi de paramètres physiques du moteur (température, vibrations, vitesse de rotation) durant une phase d’essai. Une fois ces données recueillies, le technicien doit ensuite les reporter manuellement dans un rapport de diagnostic au format papier ou PDF.

Cette double saisie – collecte des mesures d’un côté, remplissage manuel du rapport de l’autre – est chronophage et source d’erreurs de transcription. Il n’existe pas, à ce jour, d’outil permettant de centraliser l’ensemble de ces données, de les analyser automatiquement et de générer directement le rapport final.

2.2 Objectifs

L’objectif de ce projet est de concevoir une solution matérielle et logicielle permettant de faciliter et fiabiliser le diagnostic des moteurs électriques par les techniciens. Plus précisément, le système doit permettre de :

- Saisir les informations générales du moteur testé ;
- Saisir les mesures manuelles effectuées par le technicien (courant par phase, résistance d’isolement) ;
- Acquérir automatiquement, via un microcontrôleur et des capteurs dédiés, la température, les vibrations et la vitesse de rotation (RPM) du moteur pendant une phase d’essai ;

- Analyser l'ensemble des données collectées pour aider le technicien à établir son diagnostic ;
- Générer automatiquement un rapport PDF, reprenant la trame du rapport actuellement rempli à la main par les techniciens.

2.3 Méthodologie

Le projet a été découpé en deux grandes parties complémentaires, développées en parallèle puis intégrées progressivement :

- Une partie **matérielle (hardware)** : conception d'un système embarqué à base de microcontrôleur **STM32**, équipé de trois capteurs (température, vibration, vitesse de rotation), chargé de l'acquisition des données pendant l'essai du moteur ;
- Une partie **logicielle (software)** : développement d'une application de bureau en **HTML, CSS et JavaScript**, à l'aide du framework **Electron**, communiquant avec le microcontrôleur en **UART**, et assurant la saisie des données, l'analyse et la génération automatique du rapport PDF.

La démarche suivie s'est déroulée en plusieurs étapes :

1. **Analyse du besoin** : observation du processus de diagnostic existant à l'atelier de bobinage, et identification des mesures effectuées manuellement par les techniciens ;
2. **Choix du matériel** : sélection du microcontrôleur et des capteurs adaptés aux grandeurs à mesurer (température, vibration, vitesse de rotation) ;
3. **Câblage et tests unitaires** : mise en place du montage sur plaque d'essai et validation individuelle de chaque capteur avant intégration ;
4. **Développement du firmware** : configuration des périphériques (CubeMX), programmation de la lecture des capteurs et de l'envoi des trames UART (CubeIDE) ;
5. **Développement de l'application** : réalisation des différents écrans (connexion, moteur, inspection, détails, historique), de la communication UART, du stockage des données et de la génération du rapport PDF ;
6. **Tests d'intégration** : essais complets du système sur des moteurs réels, du montage des capteurs jusqu'à la génération du rapport final.

Chapitre 3

Travail réalisé

3.1 Architecture générale du système

Le système développé repose sur deux blocs principaux reliés par une liaison série UART :

- Un **microcontrôleur STM32**, relié aux trois capteurs, chargé de lire leurs valeurs et de les transmettre à l'application ;
- Une **application de bureau (Electron)**, exécutée sur PC, qui se connecte au microcontrôleur, pilote l'acquisition, saisit les mesures manuelles, analyse les données et génère le rapport final.

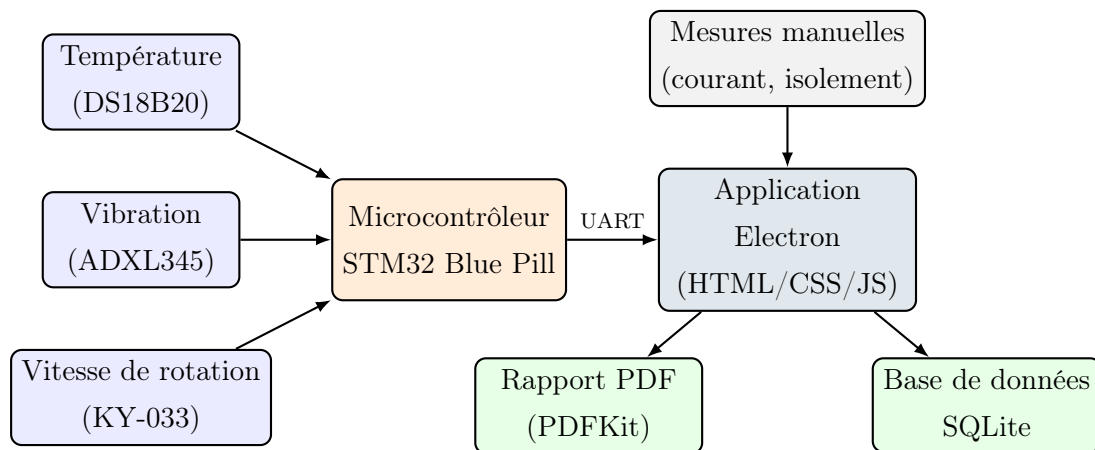


FIGURE 3.1 – Architecture générale du système (capteurs + STM32 + application Electron)

3.2 Partie matérielle

Choix du microcontrôleur

Le microcontrôleur utilisé est une carte **STM32 Blue Pill**, construite autour du composant **STM32F103C8T6** (cœur ARM Cortex-M3). Ce choix a été motivé par plusieurs raisons :

- Un **coût réduit** et une grande disponibilité, adaptés à un projet de prototypage ;
- Un nombre suffisant de **périphériques** (UART, timers, entrées/sorties numériques, ADC...) pour piloter les trois capteurs et assurer la communication série avec l'application ;
- Une **compatibilité totale** avec les outils STM32CubeMX et STM32CubeIDE utilisés pour la configuration et la programmation ;
- Un **format compact**, facile à intégrer sur plaque d'essai pour la phase de prototypage.



FIGURE 3.2 – Carte STM32 Blue Pill (STM32F103C8T6) utilisée pour le système embarqué

Capteurs

Le système d'acquisition embarqué est construit autour du microcontrôleur STM32 Blue Pill, auquel sont reliés trois capteurs :

- **DS18B20** : capteur de température numérique, utilisé pour mesurer la température du moteur pendant l'essai ;
- **ADXL345** : accéléromètre numérique, utilisé pour mesurer les vibrations du moteur ;
- **Capteur infrarouge KY-033** : utilisé pour mesurer la vitesse de rotation (RPM) du moteur.

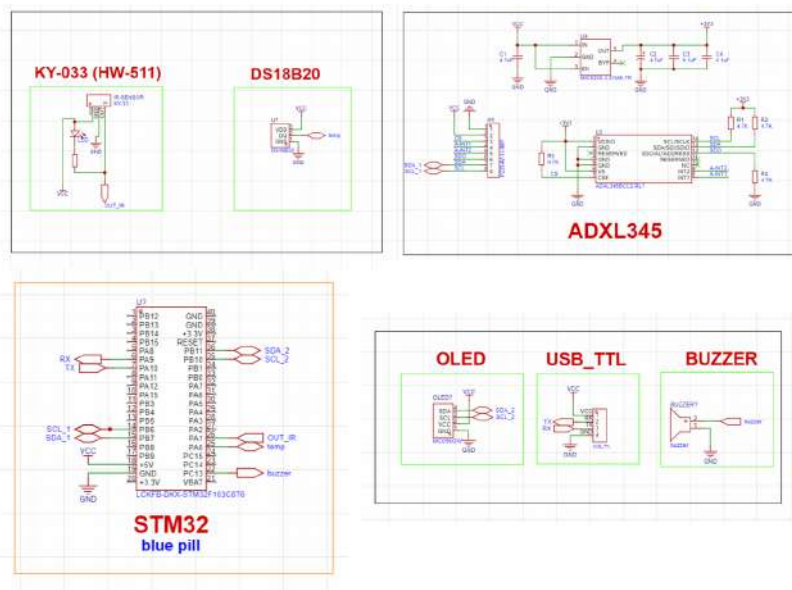


FIGURE 3.3 – Prototype du système embarqué réalisé (STM32 Blue Pill + capteurs)



FIGURE 3.4 – Capteurs utilisés : KY-033 (vitesse de rotation), DS18B20 (température), ADXL345 (vibration)

Autres composants du boîtier

En plus des trois capteurs, le boîtier réalisé intègre deux composants complémentaires, reliés eux aussi au STM32 :

- Un **buzzer**, utilisé principalement à des fins de **débogage** (signalisation sonore de certains événements ou erreurs lors du développement et des tests du système) ;
- Un **écran OLED**, intégré au boîtier, permettant d'**afficher directement sur place** les principales informations du système (état de la connexion, valeurs des capteurs...), sans nécessiter de regarder l'application sur PC.



FIGURE 3.5 – Boîtier du prototype

Le STM32 lit en continu les valeurs de ces trois capteurs et les transmet, une fois par seconde, à l'application via la liaison série UART.

Placement des capteurs et tests sur moteur réel

Chaque capteur a été positionné sur le moteur de manière à garantir une mesure fiable et représentative de son état de fonctionnement :

- Le capteur de température **DS18B20** a été fixé au contact du carter du moteur, au niveau d'une zone représentative de l'échauffement du bobinage, afin de suivre l'évolution de la température pendant l'essai ;
- L'accéléromètre **ADXL345** a été fixé rigidement sur le corps du moteur (au niveau du palier), de façon à capter fidèlement les vibrations générées par la rotation de l'arbre ;
- Le capteur infrarouge **KY-033** a été positionné en vis-à-vis de l'arbre moteur, face à un repère réfléchissant collé sur l'axe : chaque passage du repère devant le capteur génère une impulsion, dont la fréquence permet de calculer la vitesse de rotation (RPM) du moteur.

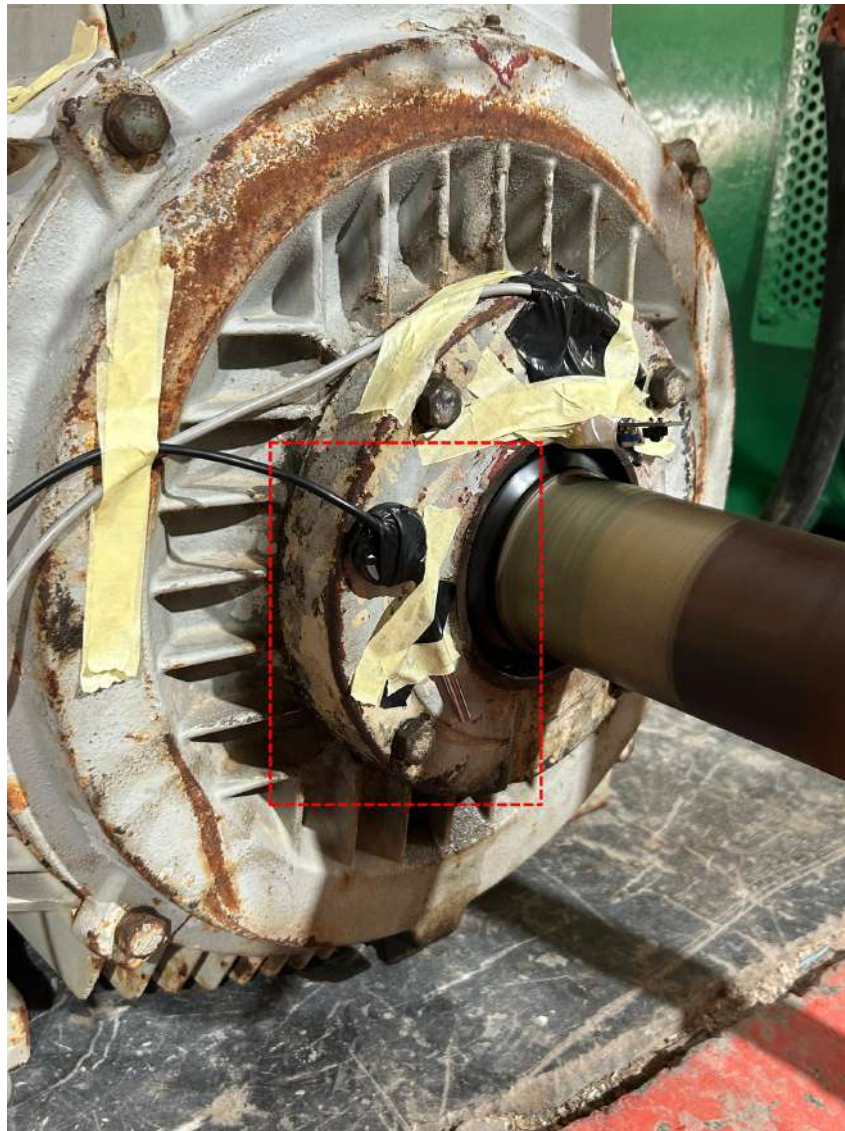


FIGURE 3.6 – Emplacement du capteur de température DS18B20 sur le moteur

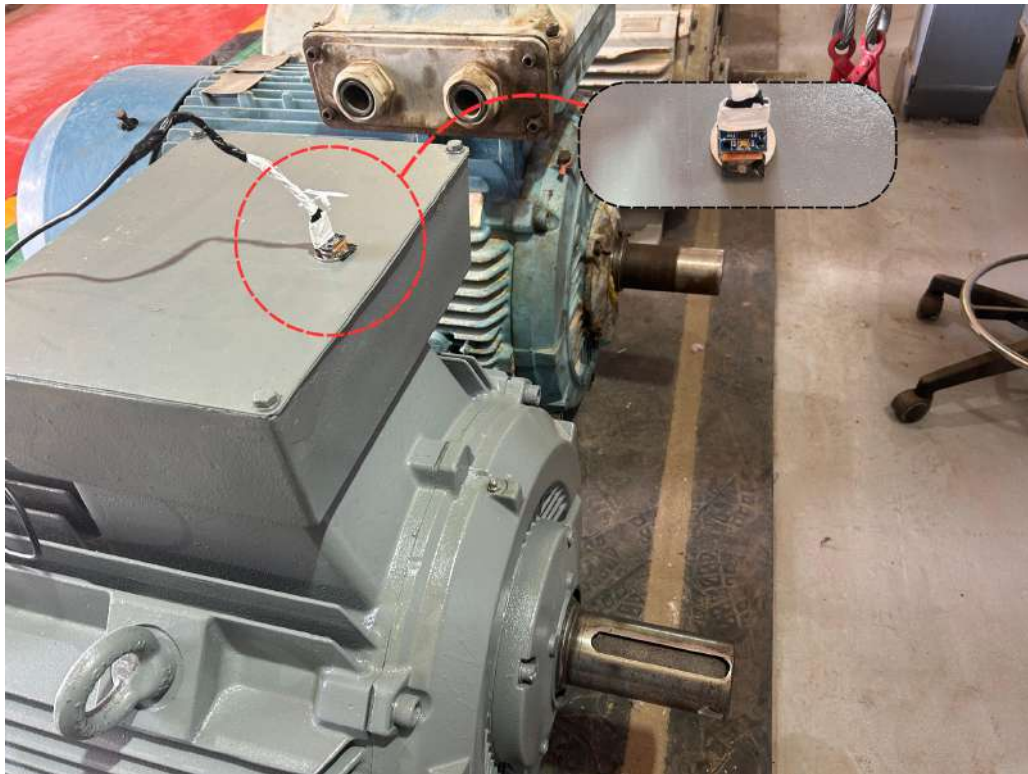


FIGURE 3.7 – Emplacement de l'accéléromètre ADXL345 pour la mesure des vibrations

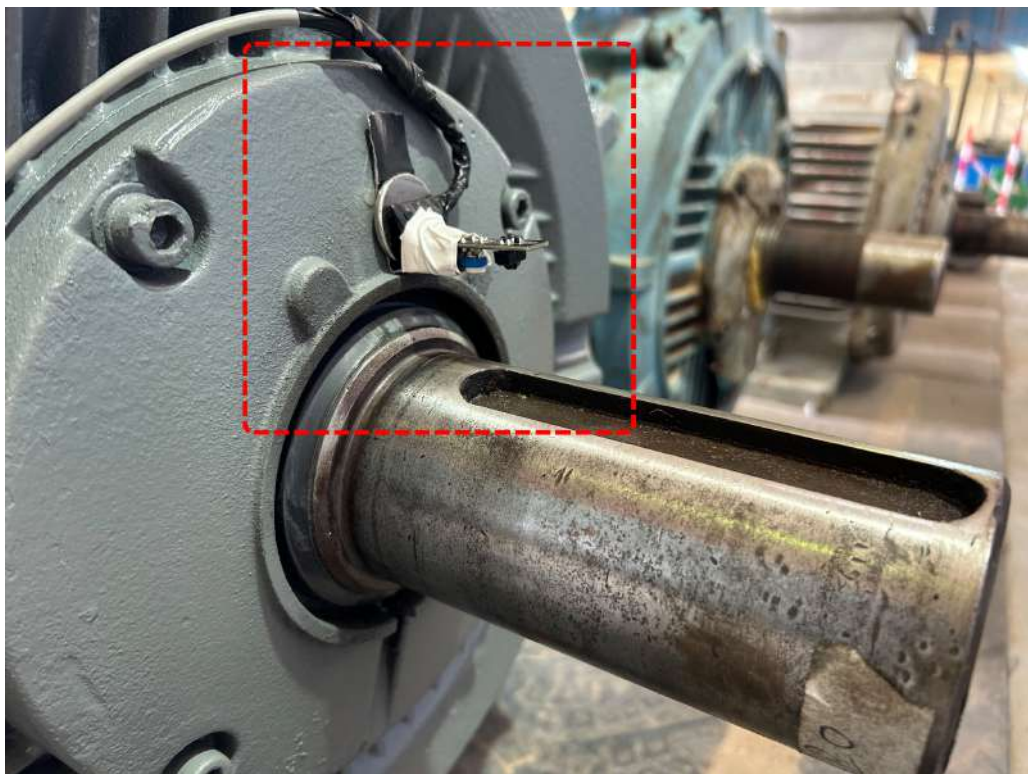


FIGURE 3.8 – Positionnement du capteur infrarouge KY-033 face au repère de l'arbre pour la mesure du RPM

Le système a ensuite été testé sur un moteur réel de l'atelier, en le laissant fonctionner

pendant la phase d'acquisition de 60 secondes, afin de valider le bon fonctionnement de l'ensemble de la chaîne de mesure (capteurs → STM32 → application). Une vidéo du test est disponible à l'adresse suivante : https://drive.google.com/drive/u/0/folders/1AaXh50N0zW4D9l5Si__vS2PoLs3jMpKc.



FIGURE 3.9 – Test du prototype sur un moteur réel de l'atelier

En complément des mesures automatiques, deux mesures manuelles sont réalisées par le technicien à l'aide d'appareils de mesure dédiés, puis saisies dans l'application :

- Le **courant par phase** du moteur ;
- La **résistance d'isolement** des enroulements, mesurée à l'aide d'un testeur d'isolement Megger MIT1025.



FIGURE 3.10 – Testeur d'isolement Megger MIT1025 utilisé pour la mesure de la résistance d'isolement

Programmation du microcontrôleur

La configuration des périphériques du STM32 (entrées/sorties, liaison UART, timers pour l'acquisition périodique, etc.) a été réalisée avec **STM32CubeMX**, qui génère automatiquement le code d'initialisation correspondant. Le développement du programme embarqué (lecture des capteurs, mise en forme des trames envoyées en UART) a ensuite été effectué sous **STM32CubeIDE**. Le programme a été téléversé sur la carte à l'aide d'un programmeur/débogueur **ST-Link**.



FIGURE 3.11 – Programmeur/débogueur ST-Link utilisé pour flasher le STM32

3.3 Partie logicielle

L'application a été développée en **HTML**, **CSS** et **JavaScript**, packagée avec **Electron** afin d'obtenir une application de bureau autonome, fonctionnant sous Windows. Le déroulement d'un test se fait en plusieurs étapes :

1. **Connexion au microcontrôleur** : l'application établit une connexion série UART avec la carte STM32 ;
2. **Saisie des informations du moteur** : le technicien renseigne les caractéristiques du moteur à tester (référence, puissance, etc.) ;
3. **Saisie des mesures manuelles** : courant par phase et résistance d'isolement (Megger MIT1025) ;
4. **Acquisition automatique** : une phase d'essai de **60 secondes** est lancée, durant laquelle l'application reçoit et enregistre, chaque seconde, les valeurs de température, de vibration et de RPM transmises par le STM32 ;
5. **Analyse** : une fois l'acquisition terminée, le technicien déclenche l'analyse des données collectées (mesures manuelles et données du capteur) ;
6. **Génération du rapport** : les résultats sont enregistrés et un rapport PDF est généré automatiquement, reprenant la même trame que le rapport rempli manuellement jusqu'ici par les techniciens, mais avec l'ensemble des champs déjà pré-remplis à partir des données saisies et acquises.

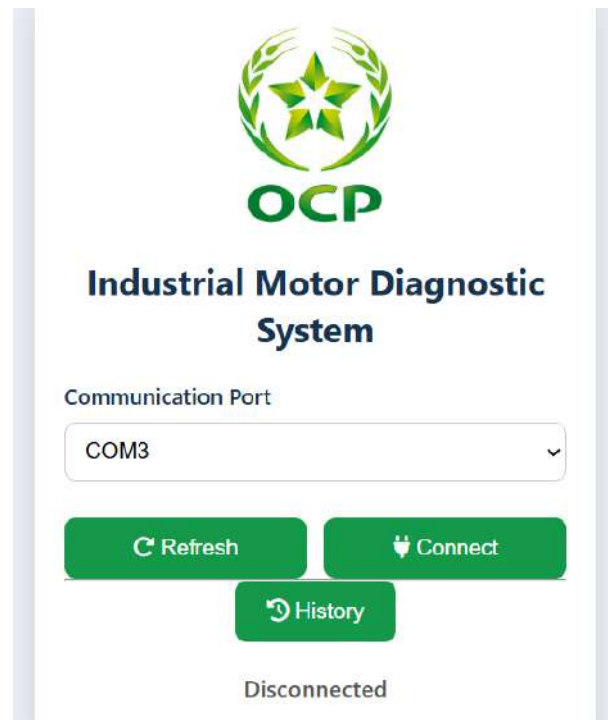


FIGURE 3.12 – Écran d'accueil : sélection du port de communication et connexion au microcontrôleur

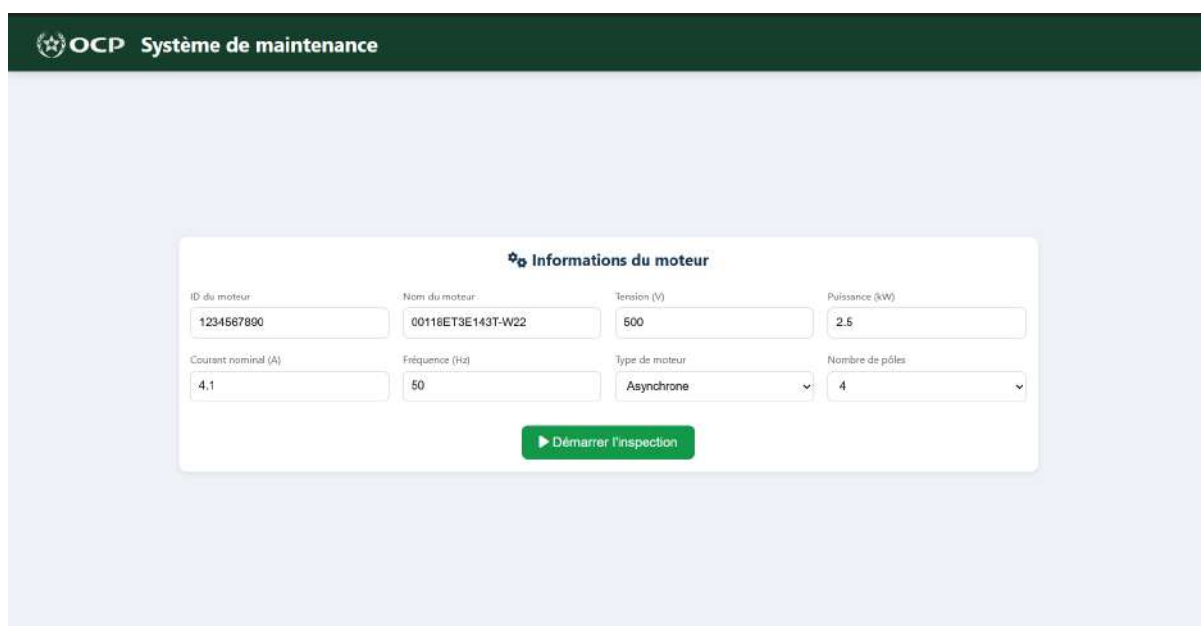


FIGURE 3.13 – Écran de saisie des informations du moteur

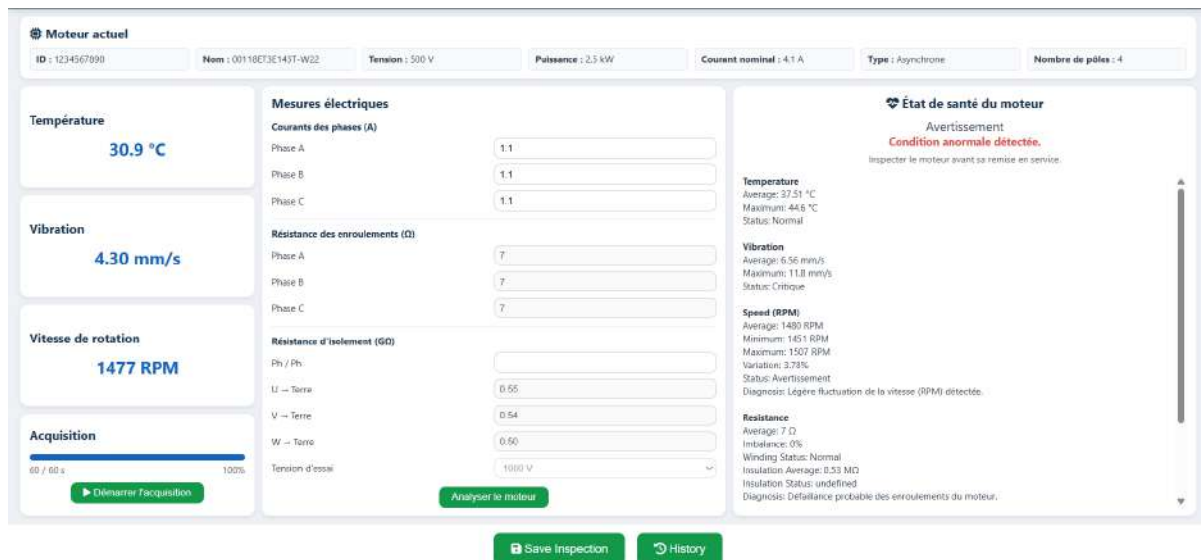


FIGURE 3.14 – Tableau de bord d'inspection : capteurs en temps réel, acquisition, mesures électriques et diagnostic

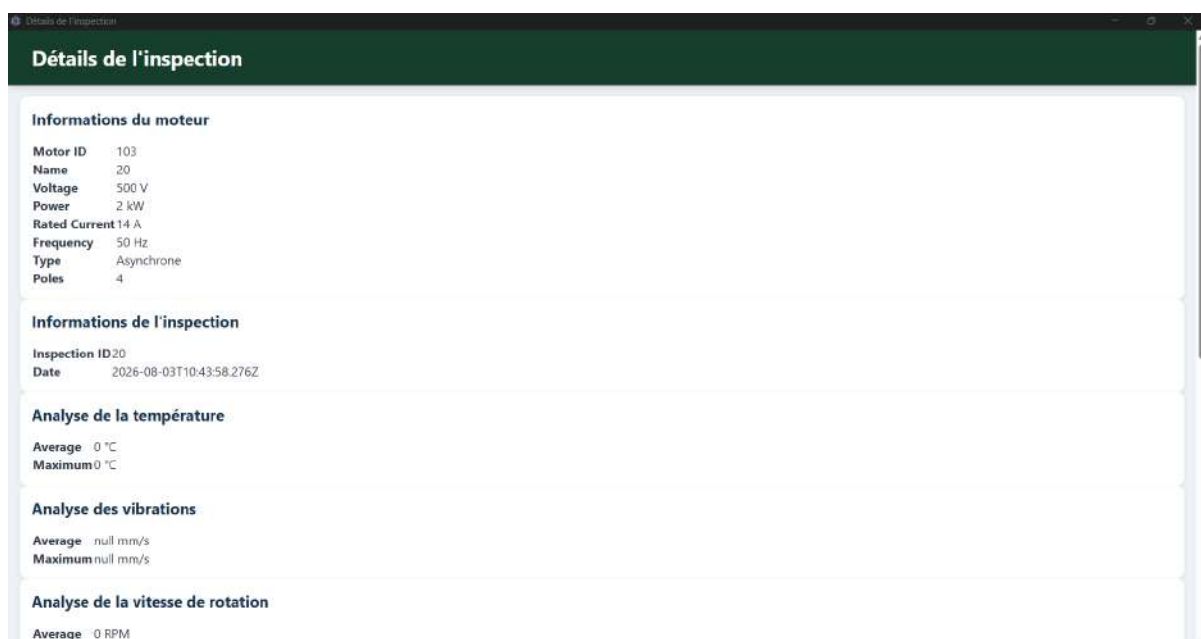
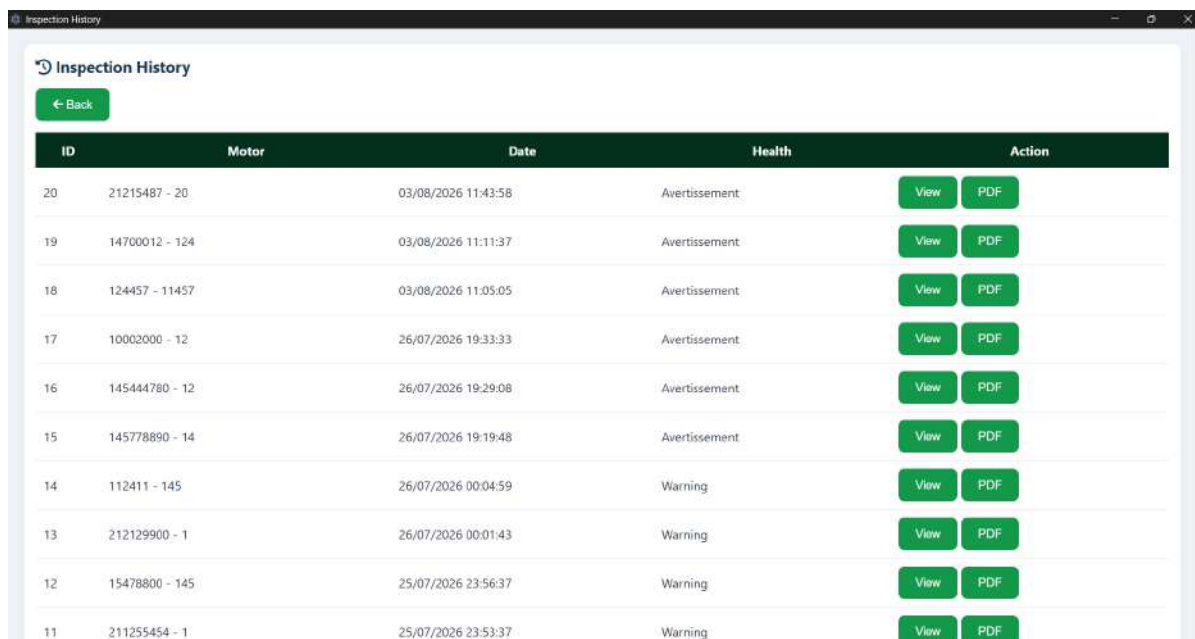


FIGURE 3.15 – Écran de détails d'une inspection : synthèse des mesures et diagnostic final



ID	Motor	Date	Health	Action
20	21215487 - 20	03/08/2026 11:43:58	Avertissement	View PDF
19	14700012 - 124	03/08/2026 11:11:37	Avertissement	View PDF
18	124457 - 11457	03/08/2026 11:05:05	Avertissement	View PDF
17	10002000 - 12	26/07/2026 19:33:33	Avertissement	View PDF
16	145444780 - 12	26/07/2026 19:29:08	Avertissement	View PDF
15	145778890 - 14	26/07/2026 19:19:48	Avertissement	View PDF
14	112411 - 145	26/07/2026 00:04:59	Warning	View PDF
13	212129900 - 1	26/07/2026 00:01:43	Warning	View PDF
12	15478800 - 145	25/07/2026 23:56:37	Warning	View PDF
11	211255454 - 1	25/07/2026 23:53:37	Warning	View PDF

FIGURE 3.16 – Historique des inspections réalisées

Outils et bibliothèques utilisés

Plusieurs bibliothèques ont été utilisées pour développer les différentes fonctionnalités de l'application :

- **Communication UART** : la bibliothèque Node.js **serialport** est utilisée pour lister les ports disponibles, établir la connexion avec la carte STM32 et échanger des données. Les trames reçues sont découpées ligne par ligne, puis transmises à l'interface de l'application (processus de rendu Electron) au fur et à mesure de leur réception.
- **Stockage des données** : les informations des moteurs et des inspections sont enregistrées localement dans une base de données **SQLite**, via la bibliothèque **sqlite3**. Deux tables principales sont utilisées : **motors** (informations des moteurs testés) et **inspections** (mesures, résultats d'analyse et diagnostic de chaque essai), reliées entre elles et exploitées notamment pour l'écran d'historique.
- **Génération du rapport PDF** : le rapport final est généré automatiquement à l'aide de la bibliothèque **PDFKit**, qui permet de construire un document PDF reprenant la trame du rapport habituellement rempli à la main, avec l'ensemble des champs pré-remplis à partir des données saisies et acquises.

3.4 Résultats obtenus

Génération automatique de la fiche d'essai : Plutôt que de saisir manuellement les mesures, le système remplit désormais automatiquement la fiche d'essai à partir des données collectées et crée un fichier PDF(température, courant , resistance etc ..).

OCP SBU Mining Direction Industrielle Mines Khouribga Direction des Moyens Généraux	Support d'enregistrement	SER/MSEB/01/05 Version 02 Page : 1/1
	FICHE D'ESSAI MOTEUR	
	Date d'essai 7/26/2026	

Matériel:

Mle:

N° de Fab : **Marque:** 12

P: 2 kW **U :** 500 V **In :** 20 A **N :** 1469 RPM

CosØ : **Couplage :**

Scé demandeur : **AVIS :** **ORDRE :**

Date de réception :

Réparation : **interne :** 0 **externe :** 0

Mesure d'isolement

Réf appareil de mesure : 16060086

Tension d'essai : 1000 V

	mesure	unité	Valeur Réf	Observation
Ph/Ph	12	GΩ	1kΩ/1v	Acceptable
Ph/Masse	12	GΩ	1kΩ/1v	Acceptable

Mesure tension/courant

Réf appareil de mesure : UT 208 TRUE RMS

Tension d'essai : 500 V

	mesure	unité	Valeur Réf	Observation
I01	20	A	In/3 < I < 2xIn/3	Élevé
I02	20	A	In/3 < I < 2xIn/3	Élevé
I03	20	A	In/3 < I < 2xIn/3	Élevé

Mesure des Résistances

Réf appareil de mesure : C.A 6240

	mesure	unité	Valeur Réf	Observation
R12	11	Ω	-	Normal
R23	11	Ω	-	Normal
R31	11	Ω	-	Normal

Mesure température paliers après min de mise en service

Réf unité de mesure : DS18B20

	mesure	unité	Valeur Réf	Observation
Coté Accouplement	25.1	°C	<70	Normal
C.O.A	26.4	°C	<70	Normal

Vitesse en tr/min

1469

Vibration en mm/s

0.97

Equipement conforme : Oul ☐ Non ☐

Date & Visa

Avertissement

Inspecter le moteur avant sa remise en service.

FIGURE 3.17 – PDF généré après l'essai

Conclusion générale

Ce stage d'initiation, effectué au sein de l'atelier de bobinage de l'OCP Khouribga, nous a permis de découvrir concrètement le monde de la maintenance industrielle et de mettre en application nos connaissances en génie des systèmes électroniques, informatique et réseaux à travers un projet complet, alliant conception matérielle et développement logiciel.

Sur le plan technique, ce stage nous a permis d'acquérir de nouvelles compétences, notamment dans la configuration et la programmation d'un microcontrôleur STM32 (STM32CubeMX, STM32CubeIDE), l'interfaçage de capteurs (température, vibration, vitesse de rotation), la communication série UART, ainsi que le développement d'une application de bureau avec Electron, incluant la gestion d'une base de données SQLite et la génération automatique de rapports PDF. Nous avons également approfondi nos connaissances sur le fonctionnement des moteurs électriques et leurs principales défaillances (roulements, isolement, bobinage).

Ce projet nous a aussi confrontés à plusieurs difficultés, notamment la synchronisation entre l'acquisition en temps réel et l'interface utilisateur, le calibrage des capteurs, la fiabilité de la communication UART, la définition des seuils de diagnostic..., que nous avons progressivement surmontées grâce à une démarche itérative de tests et d'ajustements.

Au-delà des compétences techniques, ce stage a été une réelle occasion de développer notre autonomie, notre sens de l'organisation et notre capacité à travailler en environnement industriel, au contact de techniciens expérimentés. Il nous a permis de mieux comprendre les enjeux réels de la maintenance industrielle et l'importance d'outils fiables pour accompagner les équipes techniques dans leur travail quotidien.

Enfin, plusieurs perspectives d'amélioration peuvent être envisagées pour ce projet, telles que l'ajout de nouveaux capteurs, la mise en place d'un envoi des données vers une base distante/cloud, l'intégration d'algorithmes de diagnostic plus avancés (intelligence artificielle), ou encore le déploiement du système sur d'autres ateliers de l'OCP.

Bibliographie

- [1] STMicroelectronics, *RM0008 – STM32F101/102/103/105/107 Reference Manual*, disponible sur <https://www.st.com/en/microcontrollers-microprocessors/stm32f103c8.html>.
- [2] STMicroelectronics, *STM32F103x8/xB – Datasheet*, <https://www.st.com/resource/en/datasheet/stm32f103c8.pdf>.
- [3] Analog Devices, *ADXL345 – 3-Axis, $\pm 2g/\pm 4g/\pm 8g/\pm 16g$ Digital Accelerometer*, Datasheet, <https://www.analog.com/en/products/adxl345.html>.
- [4] Analog Devices (Maxim Integrated), *DS18B20 – Programmable Resolution 1-Wire Digital Thermometer*, Datasheet, <https://www.analog.com/en/products/ds18b20.html>.
- [5] Megger, *MIT1025 – Advanced 10 kV Insulation Resistance Tester*, <https://www.megger.com/en/products/advanced-5-kv-10-kv-and-15-kv-insulation-resistance-testers>.
- [6] OpenJS Foundation, *Electron Documentation*, <https://www.electronjs.org/docs/latest/>.
- [7] *serialport – Node.js package for accessing serial ports*, <https://serialport.io/>.
- [8] *PDFKit – A JavaScript PDF generation library*, <https://pdfkit.org/>.
- [9] *SQLite – Documentation officielle*, <https://www.sqlite.org/docs.html>.
- [10] Groupe OCP, *Site officiel*, <https://www.ocpgroup.ma/>.
- [11] Commission Électrotechnique Internationale (CEI), *CEI 60034 – Machines électriques tournantes*.

Annexe A

Annexes

Cette annexe présente des extraits du code source du firmware embarqué, développé sous STM32CubeIDE en langage C. Certaines parties générées automatiquement par STM32CubeMX (initialisation des périphériques I2C, DMA, horloge...) ont été omises par souci de concision, le code présenté ci-dessous se concentrant sur la logique métier développée pour ce projet.

L'ensemble du code source du projet (firmware STM32 et application Electron) est disponible publiquement sur le dépôt GitHub suivant : <https://github.com/HrirMohamed/motor-health-monitor>.

A.1 Boucle principale (main.c)

La fonction principale initialise l'ensemble des modules (UART, capteurs, traitement des vibrations, écran OLED, gestion du protocole de communication), puis exécute une boucle infinie non bloquante.

```
1 int main(void)
2 {
3     HAL_Init();
4     SystemClock_Config();
5
6     MX_GPIO_Init();
7     MX_DMA_Init();
8     MX_I2C1_Init();
9     MX_USART1_UART_Init();
10    MX_TIM2_Init();
11    MX_I2C2_Init();
12
13    Delay_Init();
14    UART_Init();
```



```

15
16  if (ADXL345_Init() == HAL_OK)
17  {
18      UART_SendLine("ADXL345 OK");
19  }
20  else
21  {
22      UART_SendLine("ADXL345 ERROR");
23  }
24
25  if (DS18B20_Init() != HAL_OK)
26  {
27      printf("DS18B20 NOT FOUND\r\n");
28  }
29  else
30  {
31      printf("DS18B20 OK\r\n");
32      DS18B20_StartConversion();
33  }
34
35  Vibration_Init(100.0f, 1.0f, 100);  // 100Hz sampling, 1Hz HPF cutoff,
    is RMS window
36  RPM_Init();
37  OLED_Init();
38  Protocol_Init();
39
40  OLED_ShowMessage("non ", " connecte ");
41
42  /* Boucle infinie non bloquante */
43  while (1)
44  {
45      RPM_Task();
46      Protocol_Task();
47  }
48  }
49
50  void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
51  {
52      if(GPIO_Pin == GPIO_PIN_1)
53      {
54          RPM_PulseDetected();
55      }
56  }

```

Listing A.1 – Fonction principale – main.c

A.2 Communication UART (uart.c)

Ce module gère la réception des commandes envoyées par l'application (par interruption) et l'envoi des mesures vers l'application.

```

1 void UART_Init(void)
2 {
3     HAL_UART_Receive_IT(&huart1, &uartRxByte, 1);
4 }
5
6 void UART_SendString(char *text)
7 {
8     HAL_UART_Transmit(&huart1, (uint8_t*)text, strlen(text),
9         HAL_MAX_DELAY);
10 }
11
12 void UART_SendLine(char *text)
13 {
14     UART_SendString(text);
15     UART_SendString("\r\n");
16 }
17
18 uint8_t UART_CommandAvailable(void)
19 {
20     return uartCommandReady;
21 }
22
23 char* UART_GetCommand(void)
24 {
25     uartCommandReady = 0;
26     return uartCurrentCommand;
27 }
28
29 /* Interruption de reception UART : construit la commande caractere par
30    caractere,
31    jusqu'a reception du caractere de fin de ligne '\n' */
32 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
33 {
34     if(huart->Instance == USART1)
35     {
36         if(uartRxByte == '\r')
37         {
38             /* Ignore */
39         }
40         else if(uartRxByte == '\n')

```

```

39     {
40         uartCommandBuffer[uartIndex] = '\0';
41         strcpy(uartCurrentCommand, uartCommandBuffer);
42         uartIndex = 0;
43         uartCommandReady = 1;
44     }
45     else
46     {
47         if(uartIndex < sizeof(uartCommandBuffer)-1)
48         {
49             uartCommandBuffer[uartIndex++] = uartRxByte;
50         }
51     }
52
53     HAL_UART_Receive_IT(&huart1, &uartRxByte, 1);
54 }
55 }

```

Listing A.2 – Gestion de la communication UART – uart.c

A.3 Traitement du signal de vibration (Vibration.c)

Ce module transforme les échantillons d'accélération bruts fournis par l'ADXL345 (en g) en une valeur de vitesse vibratoire efficace (RMS, en mm/s), grandeur couramment utilisée pour le diagnostic vibratoire des machines tournantes. Le traitement applique un filtre passe-haut pour éliminer la gravité et la dérive, intègre l'accélération pour obtenir la vitesse, puis calcule la valeur RMS sur une fenêtre glissante.

```

1 static float HPF_Apply(HPF_t *f, float alpha, float x)
2 {
3     float y = alpha * (f->prev_out + x - f->prev_in);
4     f->prev_in = x;
5     f->prev_out = y;
6     return y;
7 }
8
9 void Vibration_Init(float sample_rate_hz, float hpf_cutoff_hz, uint16_t
    window_samples)
10 {
11     dt = 1.0f / sample_rate_hz;
12
13     /* alpha du filtre passe-haut discret, a partir de la constante de
        temps RC */
14     float rc = 1.0f / (2.0f * PI * hpf_cutoff_hz);

```

```

15     hpf_alpha = rc / (rc + dt);
16
17     window = window_samples;
18     /* ... reinitialisation des variables internes ... */
19 }
20
21 void Vibration_Update(float ax_g, float ay_g, float az_g)
22 {
23     float a[3] = { ax_g * GRAVITY_MS2, ay_g * GRAVITY_MS2, az_g *
24                   GRAVITY_MS2 };
25     float v_mag_sq = 0.0f;
26
27     for (int i = 0; i < 3; i++)
28     {
29         /* 1) suppression de la gravite / derive lente */
30         float a_ac = HPF_Apply(&hpf_accel[i], hpf_alpha, a[i]);
31
32         /* 2) integration acceleration -> vitesse (Euler) */
33         velocity[i] += a_ac * dt;
34
35         /* 3) suppression de la derive d'integration */
36         float v_ac = HPF_Apply(&hpf_vel[i], hpf_alpha, velocity[i]);
37
38         v_mag_sq += v_ac * v_ac;
39     }
40
41     sum_sq += v_mag_sq;
42     sample_cnt++;
43
44     if (sample_cnt >= window)
45     {
46         float mean_sq = sum_sq / (float)sample_cnt;
47         last_rms_mms = sqrtf(mean_sq) * 1000.0f; /* m/s -> mm/s */
48
49         sum_sq = 0.0f;
50         sample_cnt = 0;
51         ready_flag = 1;
52     }
53 }

```

Listing A.3 – Calcul de la vitesse vibratoire RMS – Vibration.c

A.4 Protocole applicatif (protocol.c)

Ce module gère la logique de haut niveau du firmware : réception des commandes envoyées par l'application (connexion, démarrage de l'acquisition), lecture périodique des capteurs et envoi des mesures, avec arrêt automatique de l'acquisition après 60 secondes.

```

1 static void ReceiveCommand(void)
2 {
3     UART_Task();
4
5     if (!UART_CommandAvailable())
6         return;
7
8     char *cmd = UART_GetCommand();
9
10    /* Handshake : confirmation de connexion avec l'application */
11    if (strcmp(cmd, "X") == 0)
12    {
13        OLED_ShowMessage("Appareil ", " connecte ");
14        Buzzer_Start(100);
15        UART_SendLine("CONNECTED");
16
17        showLogoPending = 1;
18        showLogoTime    = HAL_GetTick();
19        return;
20    }
21
22    /* Demarrage de l'acquisition (60 secondes) */
23    if (strcmp(cmd, "START") == 0)
24    {
25        if (acquisitionRunning)
26            return;
27
28        acquisitionRunning = 1;
29        startTime         = HAL_GetTick();
30        Buzzer_Start(300);
31
32        UART_SendLine("STARTED");
33        return;
34    }
35
36    UART_SendLine("UNKNOWN COMMAND");
37 }
38
39 static void SendMeasurements(void)

```

```

40 {
41     char text[50];
42
43     /* Arrêt automatique après 60 secondes */
44     if ((HAL_GetTick() - startTime) >= 60000)
45     {
46         acquisitionRunning = 0;
47         UART_SendLine("DONE");
48         Buzzer_StartTimes(3, 200);
49         return;
50     }
51
52     /* Mise à jour de la vibration toutes les 10 ms */
53     if (HAL_GetTick() - lastVibration >= 10)
54     {
55         lastVibration = HAL_GetTick();
56         if (ADXL345_ReadAcceleration(&ax, &ay, &az) == HAL_OK)
57         {
58             Vibration_Update(ax, ay, az);
59         }
60     }
61
62     /* Envoi des mesures (temperature, vibration, RPM) toutes les
        secondes */
63     if (HAL_GetTick() - lastSample >= 1000)
64     {
65         lastSample = HAL_GetTick();
66
67         if (DS18B20_IsConversionDone())
68         {
69             DS18B20_ReadTemperature(&temperature);
70             sprintf(text, "TEMP:%.2f\r\n", temperature);
71             UART_SendString(text);
72             DS18B20_StartConversion();
73         }
74
75         if (Vibration_IsReady())
76         {
77             vib = Vibration_GetVelocity_mms();
78             sprintf(text, "VIB:%.2f\r\n", vib);
79         }
80         else
81         {
82             sprintf(text, "VIB:NOT READY\r\n");
83         }

```

```

84     UART_SendString(text);
85
86     RPM = RPM_Get();
87     sprintf(text, "RPM:%0.1f\r\n", RPM);
88     UART_SendString(text);
89
90     Buzzer_Start(40);
91 }
92 }
```

Listing A.4 – Gestion des commandes et de l’acquisition – protocol.c

Remarque : la gestion non bloquante du buzzer (fonctions `Buzzer_Start`, `Buzzer_StartTimes`, `Buzzer_Update`) ainsi que les fonctions générées automatiquement par STM32CubeMX (initialisation GPIO, I2C, DMA, horloge) ne sont pas reproduites ici par souci de concision.